



Internship Assignment

Build a real-time AI experience with the Gemini API

DEADLINE

On or before Friday, 30 September 2026.

The Brief

This assignment is a chance to build something real. Pick one of the five ideas below, or bring your own, and turn it into a small working product: something we could genuinely show a client at NSOffice.

The one condition: it needs to use the Gemini API, and it should lean into something the API is uniquely good at right now, real-time voice conversation or live screen awareness, rather than a plain chatbot that could run on any model.

Build With Gemini

Google's Live API model, `gemini-3.1-flash-live-preview`, is a low-latency, audio-to-audio model built for real-time dialogue: it can listen and speak back over a live WebSocket connection, watch a shared screen or camera feed, and handle interruptions mid-response the way a real conversation does. It runs on Google AI Studio's free tier with no credit card needed.

If part of your idea needs a plain text or turn-based step alongside the live piece, `gemini-3-flash-preview` is the general-purpose free-tier model to reach for. Either way, keep the API key in an environment variable, never in the code.

Five Ideas to Choose From

Pick one of these or propose your own if it fits the brief above.

1. Live Property Walkthrough Concierge [Real Estate](#)

The user shares a listing page or floor plan on screen and talks it through out loud. Gemini watches the shared screen, answers questions on carpet area or locality comparisons as they come up and can work an EMI calculation into the conversation without breaking the flow.



2. Live Policy and Claims Voice Assistant [BFSI / Insurance](#)

A caller describes a claim or asks about a policy in plain speech. The assistant listens in real time, can be shown the policy document on screen to ground its answer in the actual clause, and speaks back a clear explanation instead of a wall of text.

3. Live Interview and Pitch Rehearsal Coach [Professional Services](#)

A candidate rehearses an interview answer, or a consultant rehearses a client pitch, out loud. Gemini Live plays the panel: it interrupts naturally with follow-up questions using barge-in, then gives a short-spoken critique at the end.

4. Live Code Review Companion [Engineering](#)

A developer shares their editor and talks through a bug as it happens. The model follows the code on screen and the conversation together and responds like a senior engineer looking over your shoulder rather than a chatbot waiting for a full question.

5. Live Health Intake Assistant [Healthcare](#)

A patient describes their symptoms out loud before a consultation. The assistant asks natural follow-up questions to fill in the gaps, then hands the doctor a structured summary, a good showcase of tool calling mid-conversation rather than just chat.

What We Need to See

- Build with the Gemini API on the free tier. Real-time voice or screen-aware use cases should run on the Live API model `gemini-3.1-flash-live-preview`; plain text or turn-based features can use `gemini-3-flash-preview`. No billing needs to be enabled on the Google Cloud project.
- Follow the NSOffice glass UI system for every screen: Electric Blue as the one accent color, DM Sans typeface, Apple-style spacing and one primary action per view. The skill folder is attached to this email, `tokens.css` and `liquid-glass.js` can be dropped straight into the project.
- Push the finished code to a public GitHub repository.
- Include a README.md with a project description, setup steps, required environment variables, and the exact commands to run it locally.
- Deploy the app on a Vercel free account so there is a working, public link to click.
- Never commit an API key. Keep it in an environment variable and add `env` to `.gitignore`.

How To Submit



- Send your response to bhargav@networkscience.ai two links: the GitHub repository and the live Vercel URL.
- A three or four line note on what you built and why you picked that idea is welcome but not required.

What We're Looking For

- Does it work end to end, live, with no local setup needed to see it in action.
- Is the real-time or screen-aware interaction genuinely used, not bolted on as an afterthought.
- Does the UI hold together and follow the brand system.
- Is the code and README clear enough for someone else to pick up.
- Original thinking beyond the five options below is welcome and counts in your favor.

